LFS256: DevOps and Workflow Management with Argo

v03.12.2024

Lab 3.1 - Installing Argo CD

In Chapter 3, we reviewed the fundamentals of Argo, exploring its capabilities and how it revolutionizes Kubernetes cluster management through GitOps. Building on that foundation, this lab marks the beginning of your hands-on journey with Argo CD, a crucial component of the Argo ecosystem.

In this lab, we'll be transitioning from theory to action. Here, you will deploy Argo CD directly into a Kubernetes cluster. This exercise is designed to provide you with practical experience and a deeper understanding of Argo CD's operational mechanics. By the end of this lab, you will have a fully functional Argo CD instance running in your cluster, serving as a cornerstone for the advanced scenarios we'll tackle in subsequent labs.

Objective

Install Argo CD, and gain access to both the Argo CD web UI and CLI.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.
- A running Kubernetes cluster.

Deploy Argo CD to Kubernetes

Create a namespace for Argo:

```
$ kubectl create namespace argocd
```

Deploy ArgoCD using the quick start manifest:

```
$ kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Verify that Argo CD is installed by running the following command:

```
$ kubectl get pods -n argocd
```

Accessing Argo CD

ArgoCD offers an intuitive and user-friendly web-based interface that simplifies the management of applications deployed on Kubernetes clusters. Below, we will explain access via the UI. Access via the CLI is also possible but explained in detail in Chapter 6.

Login via UI

In order to access Argo CD's API, you need to port forward the service. This should be done in a separate terminal window so you can run commands from your current terminal window:

```
$ kubectl port-forward svc/argocd-server -n argocd 8080:443

Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
```

Argo CD comes with a built-in admin user and auto-generated password. The password is stored as a Kubernetes secret which can be retrieved with the following command:

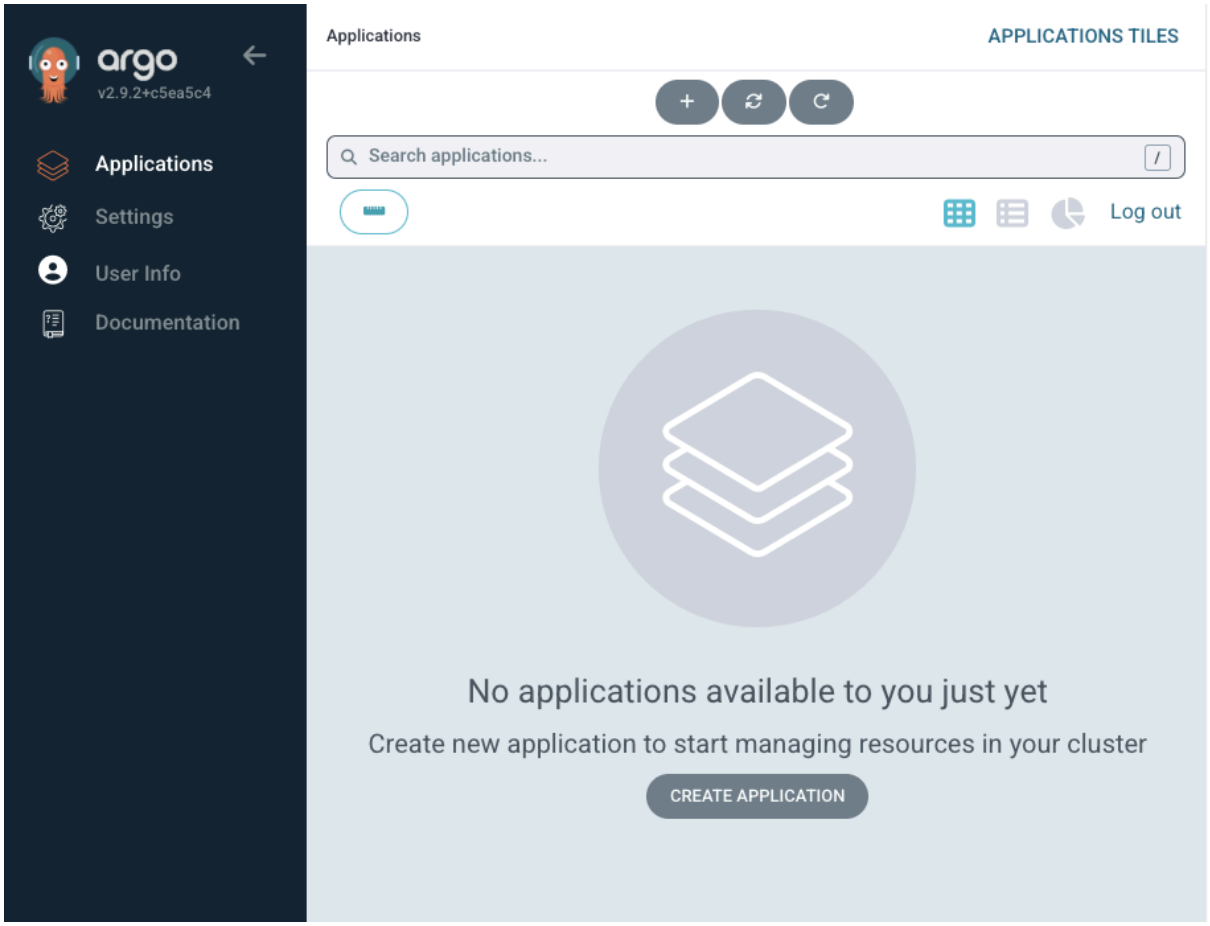
```
$ kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d; echo
```

Access the UI on https://localhost:8080. Due to the self-signed certificate, you will receive a TLS error which you will need to manually approve.

Pay close attention to the URI. It uses HTTPS and not HTTP. Navigating to http://localhost:8080 will result in a server-side error that breaks the port forwarding.

On the UI, you can login with the user **`admin`** and the auto-generated password.

Once logged in, you should see the following screen:



Screenshot of Argo CD UI after logging in

Login via CLI

Download and install the ArgoCD CLI tool suitable for your operating system and corresponding to your specific ArgoCD release version. More detailed installation instructions can be found in the CLI installation documentation.

For users on Mac, Linux, and WSL who prefer using Homebrew, you can install the tool by running:

```
$ brew install argocd
```

Using the username admin, the auto-generated password from above, and the port-forwarded service, log in to Argo CD accessible at https://localhost:8080:

```
$ argocd login localhost:8080
```

```
WARNING: server certificate had error: tls: failed to verify
certificate: x509: certificate signed by unknown authority.
```

```
Proceed insecurely (y/n)? y
Username: admin
Password:
'admin:login' logged in successfully
Context 'localhost:8080' updated
```

Upon successful login, you should see a message indicating that you're logged in to the specified ArgoCD server URL.

Lab 3.2 - Managing Applications with Argo CD

In this lab, we're moving beyond the basics and getting into the practical implementation of Argo CD, showcasing its robust capabilities in deploying applications into a Kubernetes cluster. This session is tailored to reinforce your understanding of Argo CD by offering a hands-on experience in a real-world setting.

You'll not only deploy an application using Argo CD and GitOps principles but also learn how to modify the deployed application. Witness firsthand how Argo CD swiftly updates pods in response to changes, and understand the process of rolling back to a previously deployed version, ensuring a smooth and reliable deployment experience.

Objective

Learn to deploy an application to Kubernetes using Argo CD and GitOps principles. In this lab, you will deploy an example application.

Prerequisites

- Argo CD already installed and running in your Kubernetes cluster.
- Basic understanding of Kubernetes and Argo CD.
- Git hosting service (GitHub recommended).

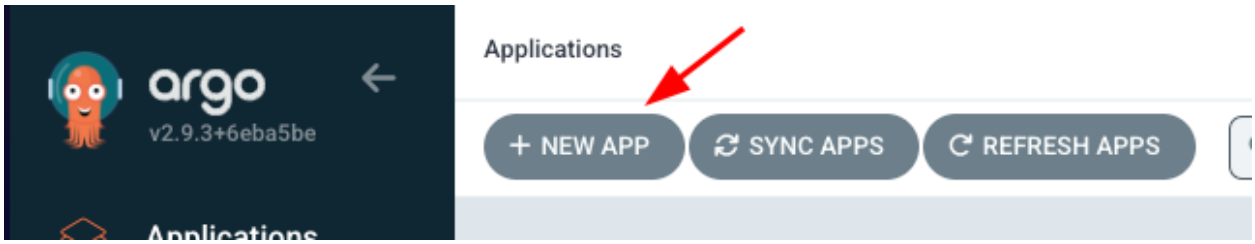
Deploy the Application

1. Fork the GitHub repository

To begin, fork the following GitHub repository: https://github.com/lftraining/LFS256-code. This step is crucial as it allows you to have your own copy of the application, enabling you to make and track changes independently during the lab.

2. Create the application in Argo CD

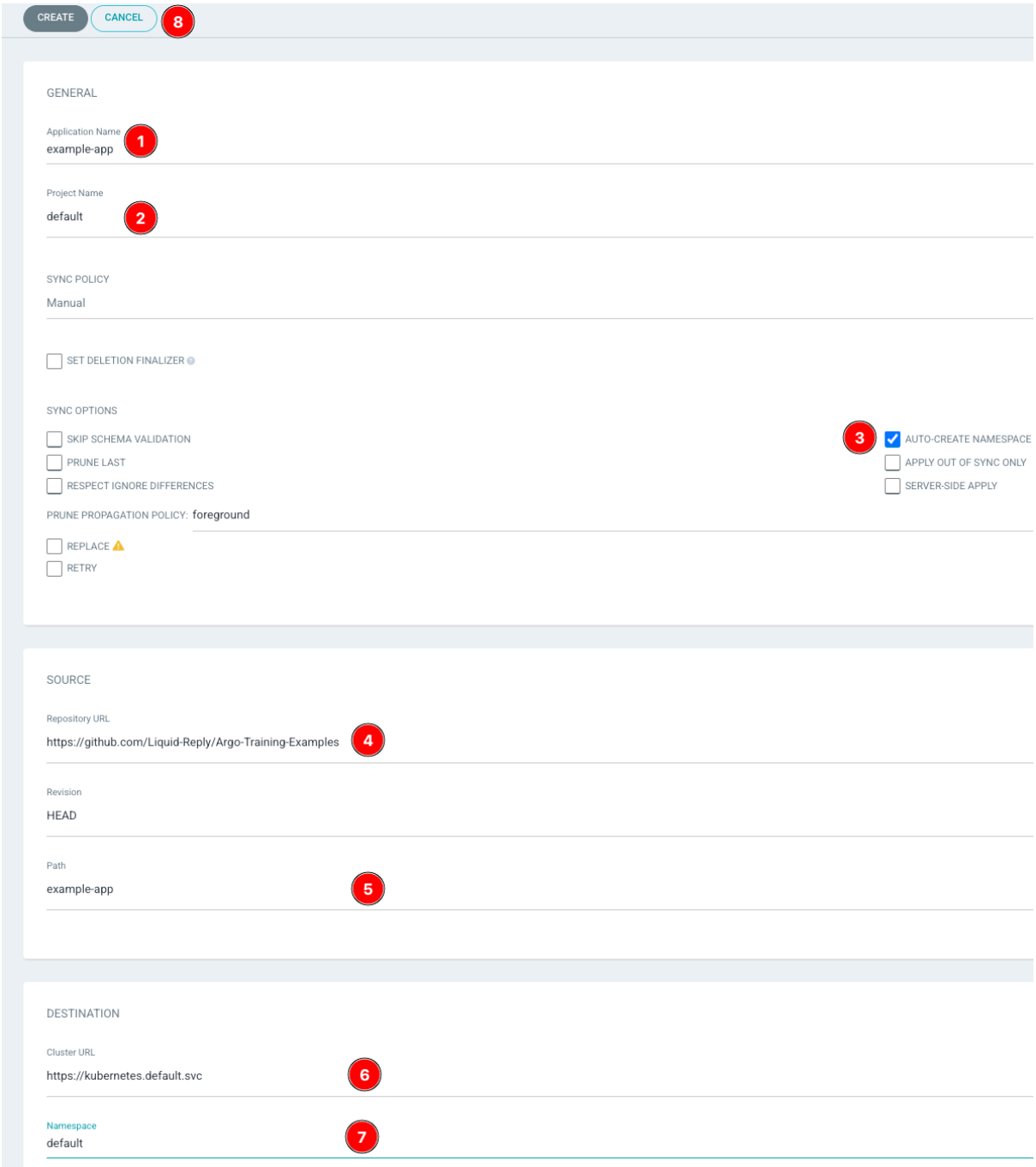
To create an Application log into the UI again by accessing https://localhost:8080 and select "+ New App".



Start the process of creating a new application

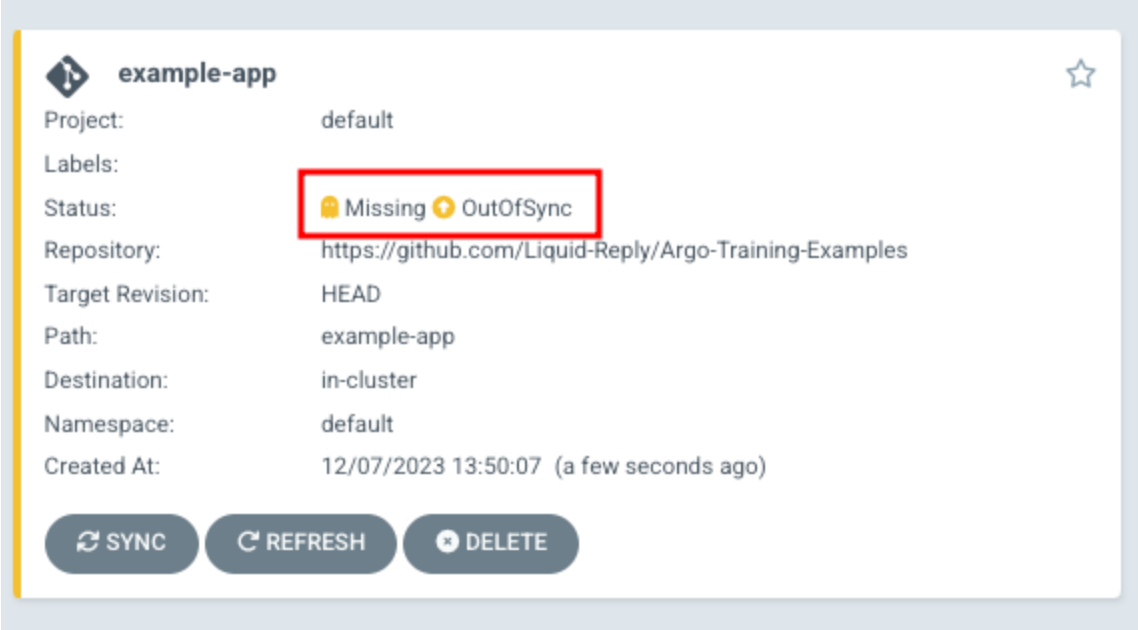
Next,

1. Write a name of your choice in "Application Name" (e.g., "example-app").
2. As Project Name choose "default".
3. Under Sync Options tick "Auto-Create Namespace".
4. In the "Source" section paste in "Repository URL" (the URL of the repository you forked in the step before).
5. In "Path" name the directory of the repository in which the YAML files are located. In this example it will be "argocd/example-app".
6. Under the "Destination" section define the "Cluster-URL". In this example we use "https://kubernetes.default.svc".
7. Define a namespace of your choice (e.g., "default").
8. Select the "Create" button at the top.



Process of creating a new application

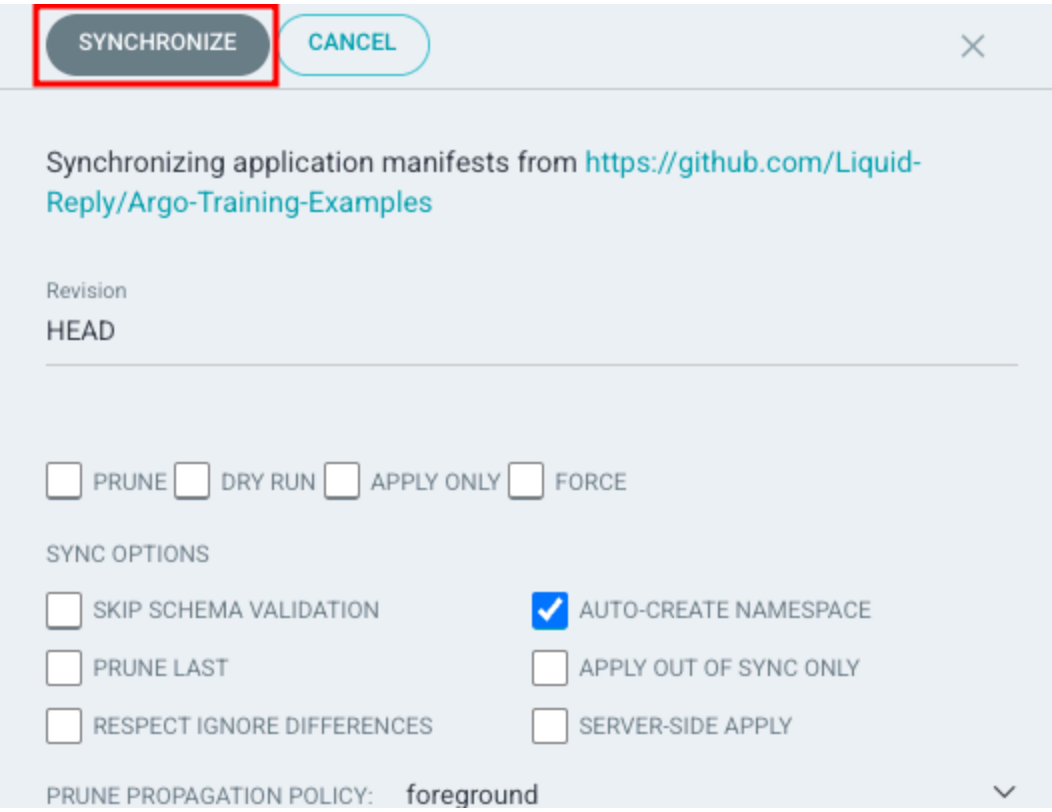
After creating the application in Argo CD, check its status using the UI. Note that the application status will be **`OutOfSync`** as it's yet to be deployed.



Undeployed application of Argo CD

3. Deploy the application

Select the “sync” button. This will open a new window. In that window you have several options to synchronize the application. For now we only have to choose “Synchronize”. The status should change to Synced and Progressing or Healthy.



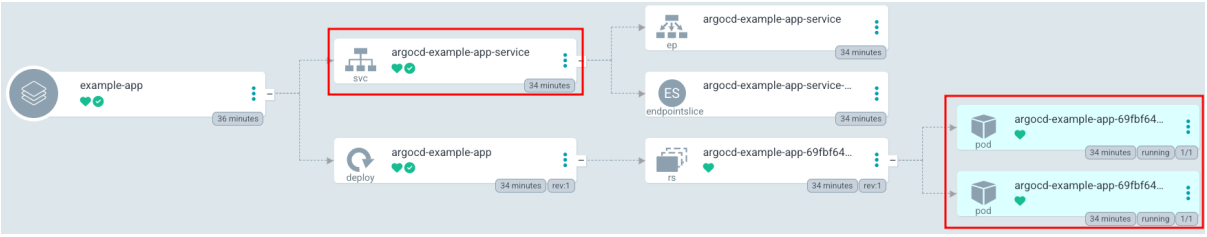
Synchronizing an application

4. Access the deployed application

Select the newly created application. In the ensuing window, you'll gain insights into its resource components, such as the two pods displayed on the right. The deployment configuration, as defined in the provided `deployment.yaml` file, specifies `replicas: 2`, indicating a desired state of two identical pods running concurrently.

Additionally, you can access logs, review events, and obtain a more detailed view of the current health status.

Make sure to take note of the associated service named `argocd-example-app-service`. This service is crucial for accessing the application through your browser. Activate port forwarding for this service to seamlessly interact with the application in your local browser environment.



Created resources for the new application

You can forward the port of the service by running the following command:

```
$ kubectl port-forward svc/argocd-example-app-service 9090:3000
```

```
Forwarding from 127.0.0.1:9090 -> 3000
Forwarding from [::1]:9090 -> 3000
```

Open http://localhost:9090 in your browser to view the application. You should see on your screen “Application Version: 1.0.0”.

Created application in the browser

5. Update the Image

In your forked repository, navigate to the deployment YAML file (`deployment.yaml`). Locate the section where the image is specified, and change the tag to a new version. If the current image tag is `liquidreply/argocd-example-app:1`, update it to `liquidreply/argocd-example-app:2`.

Commit and push the changes.

You will see the status is now `OutOfSync`. To sync it, repeat step 4. It's also possible to activate AutoSync.

Choose the application. At the top left, select “App Details”. In the new window, you will find a button “Enable Auto-Sync”. Choose it and confirm it.

If you now change the image again or anything else in the yaml file it will automatically sync the application.

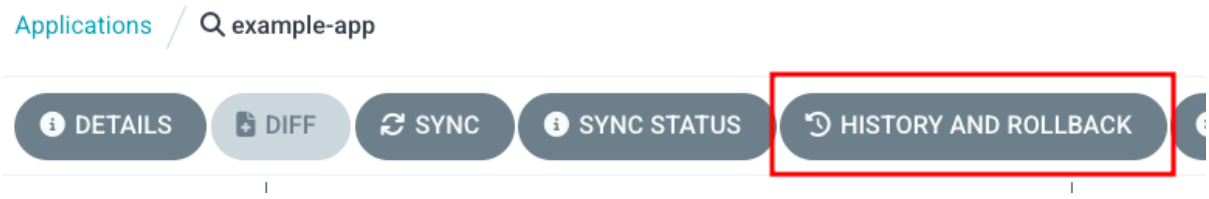
After synchronization you have to restart the port forwarding. Open http://localhost:9090 and you should be able to see “Application Version: 2.0.0”.

Updated application in the browser

6. Rollback

Now, let's delve into Argo CD's rollback capability. After updating your application, we'll guide you through effortlessly reverting to the previous version.

In the UI, you will find inside the application a button labeled “History and Rollback”.



Menu of an application

If you select it, a new window will open. In this window, you can see the history of deployment for your application. Besides information about the deployment itself, you will also be able to see a Kebab menu. Select it and choose to rollback to that deployment.

Lab 3.3 - Argo CD Security and RBAC

Security is paramount, and in this lab, we explore how to configure authentication and authorization in Argo CD. Implement Role-Based Access Control (RBAC) to finely manage user permissions.

Objective

Learn how to finely manage user permissions and ensure secure access to Argo CD resources.

Prerequisites

- Argo CD already installed and running in your Kubernetes cluster.
- Basic understanding of Kubernetes and Argo CD.

Configure Authentication and Authorization

1. Create a new user

Open the ConfigMap for editing:

```
kubectl edit cm argocd-cm -n argocd
```

Inside the ConfigMap, find or create the **`data`** section, then add a new user:

```
data:
  accounts.developer: login
```

The user “Developer” has now been created. To log in, it needs a password. You can configure it byy using the Argo CD CLI. Install it by running:

```
brew install argocd
```

Then set the password with this command:

```
argocd account update-password --account developer --new-password
Developer123
```

2. Define the RBAC rules

Open the ConfigMap for editing RBAC rules with this command:

```
kubectl edit cm argocd-rbac-cm -n argocd
```

In here we want to create a new policy:

```
p, role:synconly, applications, sync, */*, allow
g, developer, role:synconly
policy.default: role:readonly
```

This policy defines the following:

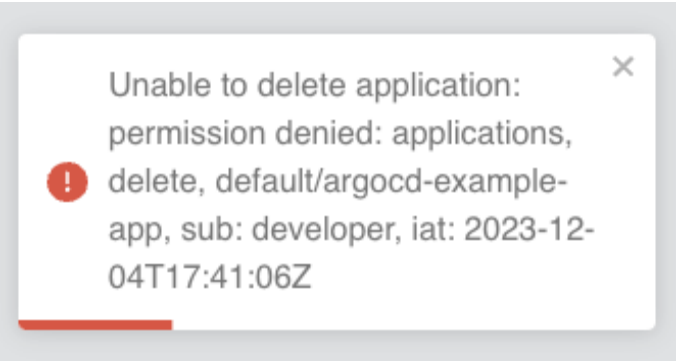
- `p, role:synconly, applications, sync, */*, allow`: This line specifies a permission rule that allows only the **`sync`** action on applications. The role **`synconly`** is created for this purpose.
- `g, developer, role:synconly`: Associates the **`developer`** group with the role **`synconly`**, granting them permission only to sync applications.
- `policy.default: role:readonly`: Specifies the default policy for users who don't fall into any explicitly defined category. In this case, they have read-only access.

In Argo CD, the RBAC rules can be intricately defined, allowing for granular control over access permissions. For a deeper understanding and detailed information on RBAC configurations, refer to the official documentation.

3. Test the new rules

Attempt to log in as the 'developer' user and evaluate the access permissions. Verify if you can successfully synchronize applications.

Additionally, explore whether actions such as deletion or other operations are restricted according to the RBAC rules assigned to the 'developer' subject.



Error after denied permission

Lab 4.1 - Installing Argo Workflows

In the previous chapter, we discussed the fundamentals of Argo Workflow, exploring its capabilities. A workflow engine designed for Kubernetes, enabling the orchestration of complex tasks and processes within containerized environments. It provides a powerful and flexible framework for defining, running, and managing workflows as a series of interconnected steps. In this chapter, we will focus on installation, accessing the graphical user interface, installing CLI, and finally exploring a few of Argo Workflow's native Kubernetes resources.

Objective

Install Argo Workflows and understand how to access Argo Workflows resources and Argo UI.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.

Install Cluster and Argo Workflows

1. Deploy Argo Workflows

Create a namespace for Argo Workflows:

```
kubectl create namespace argo
```

Deploy Argo Workflows using the quick start manifest:

```
kubectl apply -n argo -f
https://github.com/argoproj/argo-workflows/releases/download/latest/install.yaml
```

Verify that Argo Workflows is installed by running the following command:

```
$ kubectl -n argo get pods
```

```
NAME                                  READY   STATUS    RESTARTS
AGE
argo-server-5fdfcf6fd6-tfx6m          1/1     Running   0
2m
workflow-controller-5df59665c9-zz92b  1/1     Running   0
2m
```

2. Accessing UI Dashboard

Patch argo-server authentication

The Argo-server (and thus the UI) defaults to client authentication, which requires clients to provide their Kubernetes bearer token to authenticate. For more information, refer to the Argo Server Auth Mode documentation. We will switch the authentication mode to the server so that we can bypass the UI login for now:

```
kubectl patch deployment \
  argo-server \
  -n argo \
  --type='json' \
  -p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/args", "value": [
  "server",
  "--auth-mode=server"
]}]'
```

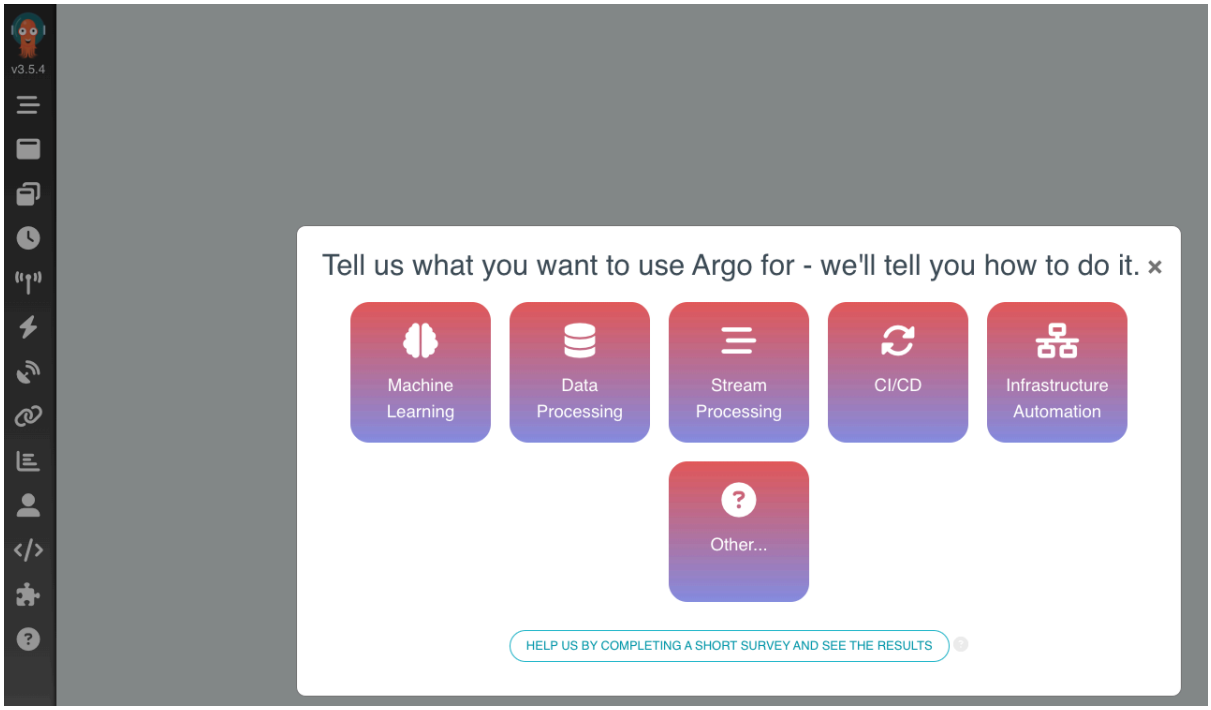
Port-forward the UI

Now you can use port-forward, so you can access the UI:

```
kubectl -n argo port-forward deployment/argo-server 2746:2746
```

The Argo Workflows UI will be accessible at https://localhost:2746. Since we are using a self-signed certificate, you need to bypass a TLS error.

Please use **HTTPS** instead of **HTTP** protocol. while navigating to Argo Workflow UI, otherwise it will cause a server-side error that will break the port forwarding.



Start Screen of Argo Workflows

3. Install the Argo Workflows CLI

Download the latest Argo Workflows version from releases. More detailed installation instructions can be found via the CLI installation documentation.

Also, verify that the **argo** CLI is installed correctly by running the following command:

```
$ argo version

argo: v3.5.2
  BuildDate: 2023-11-27T19:06:22Z
  GitCommit: 5b6ad2be163ecd3f0251a931ab84dba3c6085ad2
  GitTreeState: clean
  GitTag: v3.5.2
  GoVersion: go1.21.4
  Compiler: gc
  Platform: darwin/amd64
```

4. Optional: Enable Shell auto-completion

To get easy access to Argo Rollout Resources, the CLI can add shell completion code for several shells. For bash, you can use:

```
source <(argo completion bash)
```

Other shells are also supported. Please refer to the completion command documentation for more details.

5. Using Argo Workflows

Here is a list of the most common commands to operate with Argo Workflows:

```
argo list # list workflows
argo get <workflow-name> # display details about a workflow
argo submit <workflow-file.yaml> # submit a workflow
argo watch <workflow-name> # watch workflow progress
argo template list # list workflow templates
argo logs <workflow-name> # display logs for workflow pod(s)
```

Lab 4.2 - A Simple DAG Workflow

In this chapter, we begin our hands-on journey with Argo Workflow, creating simple workflows. This lab focuses on practical implementation, allowing users to familiarize themselves with Argo Workflow by deploying a workflow. This step-by-step exploration will create a minimal workflow demonstrating a DAG Workflow. DAGs are particularly useful in scenarios where tasks or steps have interdependencies, and their execution order needs to be carefully managed. DAGs can be highly beneficial in data processing pipelines, scientific workflows, machine learning, and multi-step deployment workflows. This example will help you understand how to define and run a simple DAG Workflow.

Objective

To see Argo Workflows in action:

- We will create a simple WorkflowTemplate that represents a task in our DAG Workflow, and
- A Workflow that uses the template and defines the relationship between the tasks.

Prerequisites

- Kubernetes cluster with argo-workflow controller.
- Argo Workflows CLI (optional).

Deploy a DAG Workflow

1. Install Resources

Here is an Argo WorkflowTemplate that runs a container. Run the following command to create a manifest called `dag-workflow-template.yaml`:

```
cat <<EOF > dag-workflow-template.yaml
```

```
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: echo-template
  namespace: argo
spec:
  serviceAccountName: argo
  templates:
  - name: echo
    inputs:
      parameters:
      - name: message
    container:
      image: alpine:3.7
      command: [echo, "{{inputs.parameters.message}}"]
EOF
```

Create a DAG workflow that uses the template and defines the relationships between tasks. Save this workflow in a file called, `dag-workflow.yaml` as shown below:

```
cat <<EOF > dag-workflow.yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  generateName: dag-diamond
  namespace: argo
spec:
  entrypoint: diamond
  templates:
  - name: diamond
    dag:
      tasks:
      - name: A
        arguments:
          parameters: [{name: message, value: A}]
        templateRef:
          name: echo-template
          template: echo
      - name: B
        dependencies: [A]
        templateRef:
          name: echo-template
          template: echo
        arguments:
```

```
        parameters: [{name: message, value: B}]
    - name: C
      dependencies: [A]
      templateRef:
        name: echo-template
        template: echo
      arguments:
        parameters: [{name: message, value: C}]
    - name: D
      dependencies: [B, C]
      templateRef:
        name: echo-template
        template: echo
      arguments:
        parameters: [{name: message, value: D}]

EOF
```

To successfully deploy this Workflow we need to temporarily grant admin permissions to argo ServiceAccount as follows:

```
$ kubectl create rolebinding default-admin --clusterrole=admin
--serviceaccount=argo:default -n argo
```

For a production environment, please follow the official documentation.

As the next step, create the WorkflowTemplate shown above, and run the following command:

```
kubectl apply -f dag-workflow-template.yaml
```

Then use the following command to submit the DAG workflow to your Kubernetes cluster:

```
kubectl apply -f dag-workflow.yaml
```

2. Inspect Workflow Resource

Check whether the WorkflowTemplate and Workflow resources have been created:

```
$ kubectl -n argo get workflowtemplates.argoproj.io

NAME            AGE
echo-template   13m

$ kubectl -n argo get workflow

NAME                STATUS      AGE     MESSAGE
dag-diamond-gvr2w   Succeeded   5m20s
```

You can achieve the same by using the Argo Workflow CLI as follows:

```
$ argo -n argo list

NAME                STATUS      AGE   DURATION   PRIORITY   MESSAGE
dag-diamond-gvr2w   Succeeded   6m    40s        0
```

Now let's review the details of the Workflow that we just submitted. The output for the command below will be the same as the information shown when you submitted the Workflow:

```
$ argo -n argo get dag-diamond-gvr2w

Name:                dag-diamond-gvr2w
Namespace:           argo
ServiceAccount:      unset (will run with the default ServiceAccount)
Status:              Succeeded
Conditions:
 PodRunning          False
 Completed           True
Created:             Thu Jan 18 19:07:48 +0100 (6 minutes ago)
Started:             Thu Jan 18 19:07:48 +0100 (6 minutes ago)
Finished:            Thu Jan 18 19:08:28 +0100 (6 minutes ago)
Duration:            40 seconds
Progress:            4/4
ResourcesDuration:   13s*(1 cpu),13s*(100Mi memory)

STEP                  TEMPLATE            PODNAME
DURATION  MESSAGE
 ✔ dag-diamond-gvr2w  diamond
 ├─✔ A                echo-template/echo
dag-diamond-gvr2w-echo-3447140468  4s
 ├─✔ B                echo-template/echo
dag-diamond-gvr2w-echo-3497473325  4s
 ├─✔ C                echo-template/echo
dag-diamond-gvr2w-echo-3480695706  9s
 └─✔ D                echo-template/echo
dag-diamond-gvr2w-echo-3396807611  4s
```

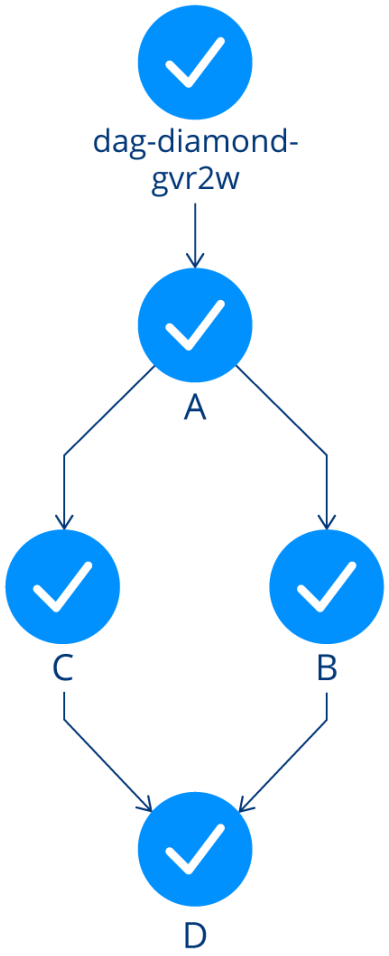
We can also check the logs of the Workflow as follows:

```
$ argo -n argo logs dag-diamond-gvr2w

dag-diamond-gvr2w-echo-3447140468: A
dag-diamond-gvr2w-echo-3447140468: time="2024-01-18T18:07:51.833Z"
level=info msg="sub-process exited" argo=true error="<nil>"
dag-diamond-gvr2w-echo-3497473325: B
```

```
dag-diamond-gvr2w-echo-3497473325: time="2024-01-18T18:08:01.922Z"
level=info msg="sub-process exited" argo=true error="<nil>"
dag-diamond-gvr2w-echo-3480695706: C
dag-diamond-gvr2w-echo-3480695706: time="2024-01-18T18:08:07.091Z"
level=info msg="sub-process exited" argo=true error="<nil>"
dag-diamond-gvr2w-echo-3396807611: D
dag-diamond-gvr2w-echo-3396807611: time="2024-01-18T18:08:21.956Z"
level=info msg="sub-process exited" argo=true error="<nil>"
```

As we can see, in this workflow, step A runs first, since it has no dependencies. After A has finished, then steps B and C run in simultaneously. When B and C are completed, step D starts. You can also view the completed steps in Argo UI as shown below.



A simple DAG workflow

3. Clean Up the Resources

We successfully created an argo Workflow. To keep our working cluster nice and clean, we are going to clean up the resources we created:

```
$ kubectl -n argo delete workflow dag-diamond-gvr2w
$ kubectl -n argo delete workflowtemplates.argoproj.io echo-template
```

4. Summary

In this exercise, we successfully implemented a lab setup by deploying DAG Workflows. Within a Kubernetes environment, we demonstrated how to define dependencies between tasks which can be well-suited for use cases that involve complex dependencies and workflows. For example, data extraction must be completed before data transformation, and transformation must finish before data loading. Argo Workflow DAG allows you to model and execute these dependencies efficiently.

Lab 4.3 - CI/CD Using Argo Workflows

This lab will give you an overview of implementing a CI/CD pipeline using Argo Workflow. Imagine you have a microservices-based application deployed on a Kubernetes cluster. You want to implement a CI/CD pipeline using Argo Workflows to automate the build, test, and deployment process for each microservice.

Objective

- Define workflows that include various steps such as building, testing, and deploying applications.
- Incorporate testing steps into your workflows to ensure the quality of your applications.
- Run unit tests, integration tests, and other relevant tests as part of the CI/CD process.

Prerequisites

- Kubernetes cluster with argo-workflow controller.
- Argo Workflows CLI (optional).

Create a CI/CD Pipeline with Argo Workflow

1. Create Argo Workflow

We can start by creating and managing our workflows as code. We will create a Python application workflow with the following steps:

- Build: The build step builds the image with the latest changes, using a Python 3 image for this scenario.
- Tests: The test step mounts a volume with test files and runs unit tests with the Python unittest library.

- Deployment: The deploy step runs the Python container and prints deploy. Normally, this step would involve pushing the tested code to a container registry, like AWS ECR or Harbor, and then deploying it to the production environment.

This lab provides a basic setup for Argo Workflow CI/CD. Depending on your specific requirements, you may want to enhance the workflow with additional steps, such as testing, linting, or pushing the Docker image to a registry.

Define the Argo Workflow manifest (`workflow-ci.yaml`) that describes the sequence of tasks and dependencies.

```
cat <<EOF > workflow-ci.yaml
apiVersion: argoproj.io/v1alpha1
kind: Workflow
metadata:
  name: python-app
spec:
  entrypoint: python-app
  templates:
  - name: python-app
    steps:
    - - name: build
        template: build
    - - name: test
        template: test
    - - name: deploy
        template: deploy
  - name: build
    container:
      image: python:3.11
      command: [python]
      args: ["-c", "print('build')"]
  - name: test
    container:
      image: python:3.11
      command: [python]
      args: ["-m", "unittest", "discover", "-s", "/app/tests"]
      volumeMounts:
      - name: test-volume
        mountPath: /app/tests
    volumes:
    - name: test-volume
      hostPath:
        path: /path/to/tests
```

```
  - name: deploy
    container:
      image: python:3.11
      command: [python]
      args: ["-c", "print('deploy')"]
EOF
```

2. Deploy the Workflow

Deploy the workflow by running the following command:

```
$ kubectl -n argo-workflows apply -f workflow-ci.yaml
```

This will create an Argo Workflow with the name **`python-app`** along with their respective templates and steps.

3. Inspect the Workflow

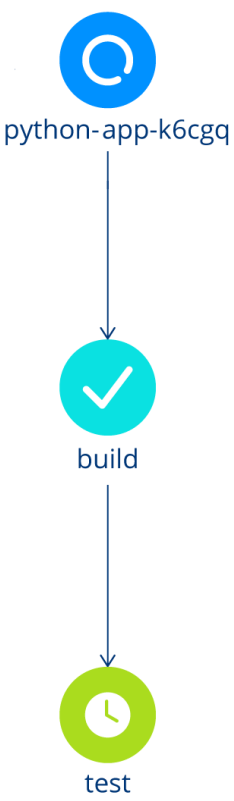
Now, let's execute the **`argo get`** command to retrieve information about the workflow, including its status:

```
$ argo -n argo-workflows get python-app
```

```
Name:                python-app
Namespace:           argo-workflows
ServiceAccount:      unset (will run with the default ServiceAccount)
Status:              Succeeded
Conditions:
 PodRunning          False
 Completed           True
Created:             Mon Jan 08 00:07:51 +0100 (21 minutes ago)
Started:             Mon Jan 08 00:07:51 +0100 (21 minutes ago)
Finished:            Mon Jan 08 00:08:39 +0100 (20 minutes ago)
Duration:            48 seconds
Progress:            3/3
ResourcesDuration:   26s*(1 cpu),16s*(100Mi memory)

STEP            TEMPLATE    PODNAME                         DURATION
MESSAGE
 ✔ python-app  python-app
 ├──✔ build   build       python-app-build-3831382643  19s
 ├──✔ test    test        python-app-test-461837862    4s
 └──✔ deploy  deploy      python-app-deploy-988129288  5s
```

You can also see the Workflow progress in Argo UI:



CI/CD Workflow progress overview

We can also view the logs:

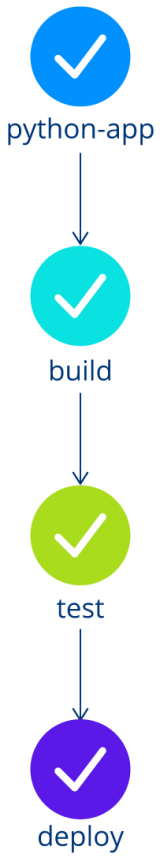
```
$ argo -n argo-workflows logs python-app

python-app-build-3831382643: time="2024-01-07T23:08:09.164Z"
level=info msg="capturing logs" argo=true
python-app-build-3831382643: build
python-app-build-3831382643: time="2024-01-07T23:08:10.165Z"
level=info msg="sub-process exited" argo=true error="<nil>"
python-app-test-461837862: time="2024-01-07T23:08:22.010Z" level=info
msg="capturing logs" argo=true
python-app-test-461837862:
python-app-test-461837862:
----------------------------------------------------------------------
python-app-test-461837862: Ran 0 tests in 0.000s
python-app-test-461837862:
python-app-test-461837862: OK
python-app-test-461837862: time="2024-01-07T23:08:23.012Z" level=info
msg="sub-process exited" argo=true error="<nil>"
```

```
python-app-deploy-988129288: time="2024-01-07T23:08:32.046Z"
level=info msg="capturing logs" argo=true
python-app-deploy-988129288: deploy
python-app-deploy-988129288: time="2024-01-07T23:08:33.047Z"
level=info msg="sub-process exited" argo=true error="<nil>"
```

Refresh the UI of Argo Workflows. You should see a new workflow that is processing.

Refresh again after a few minutes and you should see the following screen with a completed workflow.



CI/CD Workflow overview

4. Summary

In this chapter, we completed a lab setup by deploying a CI/CD pipeline using Argo Workflow to show how to establish a robust and automated pipeline for building, testing, and deploying applications in Kubernetes, promoting a DevOps culture and enhancing software delivery practices. Here we mention a few benefits of such an implementation:

- Automation and efficiency: Argo Workflows automates the CI/CD pipeline, reducing manual intervention and enhancing efficiency.

- Consistency and reproducibility: With Argo Workflows, CI/CD processes are defined declaratively, ensuring consistency across multiple runs and environments.
- Scalability: Argo Workflows provides scalability, allowing CI/CD processes to scale with the growing needs of the application.
- Visibility and monitoring: Argo Workflows' UI and CLI provide visibility into workflow execution, allowing teams to monitor progress and troubleshoot issues.
- Flexibility and customization: Argo Workflows offers flexibility in defining custom workflows, allowing teams to tailor CI/CD pipelines to their specific requirements.

Lab 5.1 - Installing Argo Rollouts

Objective

Set up a local Kubernetes cluster using Kind, install Argo Rollouts, and understand how to access Argo Rollout resources.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.

Install Cluster and Argo Rollouts

***NOTE**: Steps 1-4 might not be necessary if you already followed the setup during a previous chapter.*

1. Installing Docker

Ensure Docker is installed and running on your machine.

- Installation instructions can be found on the Docker website.

2. Installing Kind

Download and install Kind following the instructions from the Kind official website.

3. Creating a Kubernetes Cluster with Kind

Create a cluster by running the following command:

```
kind create cluster
```

This command creates a single-node Kubernetes cluster running inside a Docker container.

4. Installing kubectl

Instructions for downloading kubectl can be found in the Kubernetes official documentation.

5. Deploy Argo Rollouts

Create a namespace for Argo Rollouts:

```
kubectl create namespace argo-rollouts
```

Deploy Argo Rollouts using the quick start manifest:

```
kubectl apply -n argo-rollouts -f
https://github.com/argoproj/argo-rollouts/releases/download/v1.6.4/ins
tall.yaml
```

This will install custom resource definitions as well as the Argo Rollouts controller.

During this course we use Argo Rollouts in version 1.6.4. We recommend using the same version to ensure consistent results.

Verify that Argo Rollouts is installed by running the following command:

```
kubectl get pods -n argo-rollouts
```

6. Install Rollouts kubectl Plugin

Download the latest Argo Rollouts kubectl Plugin version from https://github.com/argoproj/argo-rollouts/releases/latest/. More detailed installation instructions can be found via the CLI installation documentation.

Also available in Mac, Linux and WSL Homebrew:

```
brew install argoproj/tap/kubectl-argo-rollouts
```

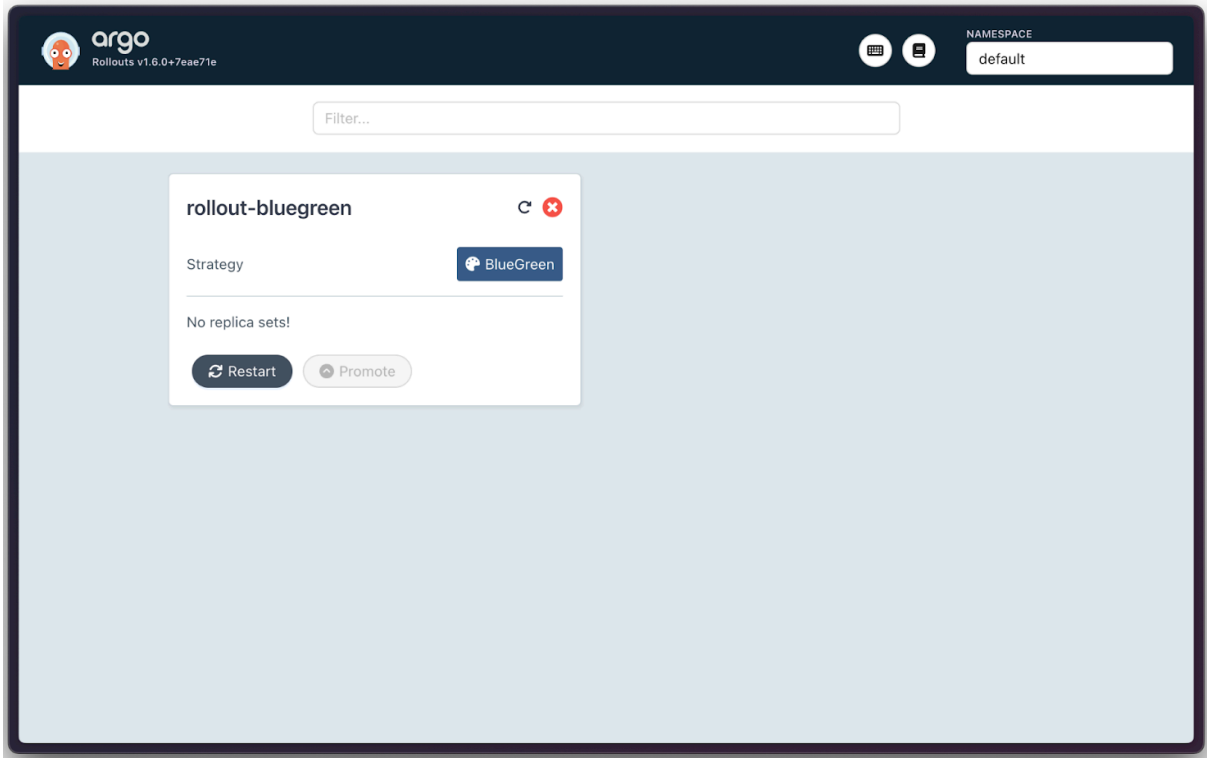
Verify that the `argo` CLI is installed correctly by running the following command:

```
kubectl argo rollouts version
```

7. UI Dashboard

For the sake of completeness it needs to be mentioned that Argo Rollouts ships with a fully fledged UI Dashboard. It can be accessed via **`kubectl argo rollouts dashboard`** and provides a nice overview and basic commands for administration.

As the Dashboard is self-explanatory, we will not discuss it in detail during this course.



The Argo Rollouts Dashboard displaying a sample app "rollout-bluegreen"

***NOTE**: If no rollout resources are in place, the dashboard will display "Loading…".*

8. Optional: Shell Auto-Completion

To get easy access to Argo Rollout resources, the CLI can add shell completion code for several shells. For bash, you can use:

```
source <(kubectl-argo-rollouts completion bash)
```

Other shells are supported as well. Please refer to the completion command documentation for more details.

9. Using Argo Rollouts

Here is a list of the most common commands to operate with Argo Rollouts:

```
kubectl get rollout

kubectl argo rollouts get rollout

kubectl argo rollouts promote

kubectl argo rollouts undo
```

Lab 5.2 - Argo Rollouts Blue-Green

Let's dig into Argo Rollouts by creating a blue-green deployment scenario. It enables us to verify a version upgrade before the live traffic hits our service. It is easy to understand and therefore one of the most commonly used ways to roll out new versions of software without any downtime.

Objective

This lab aims to give a reader an idea of the "look and feel" of Argo Rollouts. It will demonstrate how to realize a simple blue-green scenario with Argo Rollouts. As blue-green is the most basic deployment pattern that rollout supports, this is a great intro to get a feeling of Argo Rollouts behavior.

Prerequisites

- Kubernetes cluster with argo-rollouts controller
- kubectl with argo-rollouts plugin (optional)

Creating Blue-Green Deployments with Argo Rollouts

1. Install Resources

For the beginning, let's check for existing rollouts.

```
$ kubectl get rollout
```

```
No resources found in default namespace.
```

As expected, there are no rollouts (yet) to be found in our cluster. Let's create one:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
```

```
metadata:
  name: rollout-bluegreen
spec:
  replicas: 2
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: rollout-bluegreen
  template:
    metadata:
      labels:
        app: rollout-bluegreen
    spec:
      containers:
      - name: rollouts-demo
        image: argoproj/rollouts-demo:blue
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
  strategy:
    blueGreen:
      activeService: rollout-bluegreen-active
      previewService: rollout-bluegreen-preview
      autoPromotionEnabled: false
EOF
rollout.argoproj.io/rollout-bluegreen created
```

Check whether the Rollout resource has been created.

```
$ kubectl get rollout
```

```
NAME                DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
rollout-bluegreen   2                                            75s
```

It has been created, but is not ready yet. Let's explore the reasons behind using the Argo Rollouts kubectl plugin to see if we can better understand its functionality.

The plugin enables us to get status information of a specific rollout and can be queried in the form: **`kubectl argo rollouts get ro <rollout-name>`**.

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

```
Name:            rollout-bluegreen
Namespace:       default
Status:          ✖ Degraded
```

```
Message:         InvalidSpec: The Rollout "rollout-bluegreen" is
invalid: spec.strategy.blueGreen.activeService: Invalid value:
"rollout-bluegreen-active": service "rollout-bluegreen-active" not
found
Strategy:        BlueGreen
Replicas:
  Desired:       2
  Current:       0
  Updated:       0
  Ready:         0
  Available:     0

NAME                  KIND     STATUS       AGE  INFO
⟳ rollout-bluegreen   Rollout  ✖ Degraded   36s
```

The resource status is degraded, as not all requirements are met: If we look closely in our rollout manifest we see that we defined two services **`activeService`** and **`previewService`**. We need to make sure that the named services are available.

Let's create them:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: rollout-bluegreen-active
  name: rollout-bluegreen-active
spec:
  ports:
  - name: "80"
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: rollout-bluegreen
  type: ClusterIP
status:
  loadBalancer: {}
EOF
```

As mentioned, the rollouts resource references a second service. A so-called “preview” service. A preview service enables a preview stack to be reachable by an administrator. It does so without serving public traffic.

If we want the preview to go live, we need to "promote" the rollout. "Promotion" refers to setting a service live.

Therefore, we will create a preview service resource to be able to check our application before promotion (aka setting it live):

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: null
  labels:
    app: rollout-bluegreen-preview
  name: rollout-bluegreen-preview
spec:
  ports:
  - name: "80"
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: rollout-bluegreen
  type: ClusterIP
status:
  loadBalancer: {}
EOF
```

If we now check our rollout, we'll see a Healthy status:

```
$ kubectl argo rollouts get ro rollout-bluegreen

Name:            rollout-bluegreen
Namespace:       default
Status:          ✔ Healthy
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue (stable, active)
Replicas:
  Desired:       2
  Current:       2
  Updated:       2
  Ready:         2
  Available:     2

NAME                                       KIND        STATUS
AGE  INFO
```

```
⟳ rollout-bluegreen                              Rollout      ✔ Healthy
21h
└──# revision:1
   └──⧉ rollout-bluegreen-5ffd47b8d4             ReplicaSet   ✔ Healthy
82s  stable,active
      ├──□ rollout-bluegreen-5ffd47b8d4-hpjfn    Pod          ✔ Running
82s  ready:1/1
      └──□ rollout-bluegreen-5ffd47b8d4-vwwq9    Pod          ✔ Running
82s  ready:1/1
```

2. Perform an Upgrade

Now that we have a running application, let's try to perform a version upgrade using the blue-green method. Therefore, we'll adjust our image to deploy **`argoproj/rollouts-demo:green`** instead of **`blue`**:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: rollout-bluegreen
spec:
  replicas: 2
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: rollout-bluegreen
  template:
    metadata:
      labels:
        app: rollout-bluegreen
    spec:
      containers:
      - name: rollouts-demo
        image: argoproj/rollouts-demo:green
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
  strategy:
    blueGreen:
      activeService: rollout-bluegreen-active
      previewService: rollout-bluegreen-preview
      autoPromotionEnabled: false
EOF
```

The Rollout status moves from “Healthy” to “Paused”, indicating that a rollout is in progress and waits for further action.

Please note, that we explicitly set **`autoPromotionEnabled`** to **`false`** - we can skip the pausing phase and directly promote by setting this value to **`true`**.

```
$ kubectl argo rollouts get ro rollout-bluegreen
```

```
Name:            rollout-bluegreen
Namespace:       default
Status:          ‖ Paused
Message:         BlueGreenPause
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue (stable, active)
                 argoproj/rollouts-demo:green (preview)
Replicas:
  Desired:       2
  Current:       4
  Updated:       2
  Ready:         2
  Available:     2

NAME                                        KIND         STATUS
AGE    INFO
⟳ rollout-bluegreen                         Rollout      ‖ Paused
21h
├──# revision:2
│  └──⧉ rollout-bluegreen-75695867f          ReplicaSet   ✔ Healthy
3m57s  preview
│     ├──□ rollout-bluegreen-75695867f-lg252 Pod          ✔ Running
3m57s  ready:1/1
│     └──□ rollout-bluegreen-75695867f-n6rnr Pod          ✔ Running
3m57s  ready:1/1
└──# revision:1
   └──⧉ rollout-bluegreen-5ffd47b8d4         ReplicaSet   ✔ Healthy
12m    stable,active
      ├──□ rollout-bluegreen-5ffd47b8d4-hpjfn Pod          ✔ Running
12m    ready:1/1
      └──□ rollout-bluegreen-5ffd47b8d4-vwwq9 Pod          ✔ Running
12m    ready:1/1
```

Let's investigate the rollout a little further and check replicasets:

```
$ kubectl get replicaset
```

```
NAME                                DESIRED   CURRENT   READY   AGE
rollout-bluegreen-5ffd47b8d4        2         2         2       14m
rollout-bluegreen-75695867f         2         2         2       6m33s
```

Argo rollout created a second replicaset, which is used to manage the different pod versions. Lets promote the new version:

```
$ kubectl argo rollouts promote rollout-bluegreen

rollout 'rollout-bluegreen' promoted

$ kubectl argo rollouts get ro rollout-bluegreen

Name:            rollout-bluegreen
Namespace:       default
Status:          ✔ Healthy
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue
                 argoproj/rollouts-demo:green (stable, active)
Replicas:
  Desired:       2
  Current:       4
  Updated:       2
  Ready:         2
  Available:     2

NAME                                             KIND         STATUS
AGE    INFO
⟳ rollout-bluegreen                              Rollout      ✔ Healthy
21h
├──# revision:2
│  └──⧉ rollout-bluegreen-75695867f              ReplicaSet   ✔ Healthy
8m16s  stable,active
│     ├──□ rollout-bluegreen-75695867f-lg252     Pod          ✔ Running
8m16s  ready:1/1
│     └──□ rollout-bluegreen-75695867f-n6rnr     Pod          ✔ Running
8m16s  ready:1/1
└──# revision:1
   └──⧉ rollout-bluegreen-5ffd47b8d4             ReplicaSet   ✔ Healthy
16m    delay:15s
      ├──□ rollout-bluegreen-5ffd47b8d4-hpjfn    Pod          ✔ Running
16m    ready:1/1
      └──□ rollout-bluegreen-5ffd47b8d4-vwwq9    Pod          ✔ Running
16m    ready:1/1
```

Our new revision changed from “preview” to “stable,active” - indicating that the new revision is live. We can even see this by checking our service:

```
$ kubectl describe svc rollout-bluegreen-active

Name:              rollout-bluegreen-active
Namespace:         default
Labels:            app=rollout-bluegreen-active
Annotations:       argo-rollouts.argoproj.io/managed-by-rollouts:
rollout-bluegreen
Selector:
app=rollout-bluegreen,rollouts-pod-template-hash=75695867f
Type:              ClusterIP
IP Family Policy:  SingleStack
IP Families:       IPv4
IP:                10.96.157.227
IPs:               10.96.157.227
Port:              80  80/TCP
TargetPort:        80/TCP
Endpoints:         10.244.0.11:80,10.244.0.12:80
Session Affinity:  None
Events:            <none>
```

Note that the Selector “rollouts-pod-template-hash” has the same value as the new ReplicaSet.

We just successfully performed a deployment using blue-green methodology.

3. Perform a Rollback

Let’s assume we want to roll back from the new green to the old blue image:

```
$ kubectl argo rollouts undo rollout-bluegreen

rollout 'rollout-bluegreen' undo

$ kubectl argo rollouts get ro rollout-bluegreen

Name:            rollout-bluegreen
Namespace:       default
Status:          ‖ Paused
Message:         BlueGreenPause
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue (preview)
                 argoproj/rollouts-demo:green (stable, active)
Replicas:
  Desired:       2
```

```
  Current:        4
  Updated:        2
  Ready:          2
  Available:      2

NAME                                           KIND         STATUS
AGE  INFO
⟳ rollout-bluegreen                            Rollout      ‖ Paused
21h
├──# revision:3
│  └──⧉ rollout-bluegreen-5ffd47b8d4            ReplicaSet   ✔ Healthy
21m  preview
│     ├──□ rollout-bluegreen-5ffd47b8d4-6szq8  Pod          ✔ Running
14s  ready:1/1
│     └──□ rollout-bluegreen-5ffd47b8d4-jnvl8  Pod          ✔ Running
14s  ready:1/1
└──# revision:2
   └──⧉ rollout-bluegreen-75695867f            ReplicaSet   ✔ Healthy
13m  stable,active
     ├──□ rollout-bluegreen-75695867f-lg252    Pod          ✔ Running
13m  ready:1/1
     └──□ rollout-bluegreen-75695867f-n6rnr    Pod          ✔ Running
13m  ready:1/1
```

Note, that “undo” alone did not set the blue image active. The rollout is now again in the pausing phase, waiting for promotion of the rollout:

```
$ kubectl argo rollouts promote rollout-bluegreen

rollout 'rollout-bluegreen' promoted
```

Checking our rollout and active service once again, we see, that the selector changed back to our old ReplicaSet:

```
$ kubectl argo rollouts get ro rollout-bluegreen

Name:            rollout-bluegreen
Namespace:       default
Status:          ✔ Healthy
Strategy:        BlueGreen
Images:          argoproj/rollouts-demo:blue (stable, active)
Replicas:
  Desired:       2
  Current:       2
  Updated:       2
  Ready:         2
```

```
  Available:     2

NAME                                                KIND        STATUS
AGE    INFO
⟳ rollout-bluegreen                                 Rollout     ✔ Healthy
22h
├──# revision:3
│  └──⧉ rollout-bluegreen-5ffd47b8d4                ReplicaSet  ✔ Healthy
27m    stable,active
│     ├──□ rollout-bluegreen-5ffd47b8d4-6szq8       Pod         ✔ Running
6m32s  ready:1/1
│     └──□ rollout-bluegreen-5ffd47b8d4-jnvl8       Pod         ✔ Running
6m32s  ready:1/1
└──# revision:2
   └──⧉ rollout-bluegreen-75695867f                 ReplicaSet  •
ScaledDown  19m
```

```
$ kubectl describe svc rollout-bluegreen-active
```

```
Name:              rollout-bluegreen-active
Namespace:         default
Labels:            app=rollout-bluegreen-active
Annotations:       argo-rollouts.argoproj.io/managed-by-rollouts:
rollout-bluegreen
Selector:
app=rollout-bluegreen,rollouts-pod-template-hash=5ffd47b8d4
Type:              ClusterIP
IP Family Policy:  SingleStack
IP Families:       IPv4
IP:                10.96.157.227
IPs:               10.96.157.227
Port:              80  80/TCP
TargetPort:        80/TCP
Endpoints:         10.244.0.13:80,10.244.0.14:80
Session Affinity:  None
Events:            <none>
```

4. Clean Up Resources

We successfully used the blue-green deployment pattern to deploy an application and even performed a rollback. To keep our working cluster nice and clean, we are going to clean up resources we created:

```
$ kubectl delete rollout rollout-bluegreen
```

```
rollout.argoproj.io "rollout-bluegreen" deleted

$ kubectl delete svc rollout-bluegreen-active

rollout-bluegreen-preview
service "rollout-bluegreen-active" deleted
service "rollout-bluegreen-preview" deleted
```

Lab 5.3 - Migrating an Existing Deployment to Argo Rollouts

Chances are, that you are not starting with a fresh Kubernetes installation but already have a running cluster with deployed workloads. Argo Rollouts has this scenario in mind and provides a migration path to migrate Deployments to Rollout resources.

Objective

Migrate a vanilla Kubernetes Deployment to an Argo Rollout resource.

Prerequisites

- Kubernetes cluster with an argo-rollouts controller.
- kubectl with argo-rollouts plugin (optional).

Transitioning to Argo Rollouts

1. Preparing resources

For this lab, we will create an NGINX deployment—a task you may have already undertaken numerous times:

```
$ kubectl create deploy nginx-deployment --image=nginx --replicas=3
```

Now check our running pods and deployments:

```
kubectl get po,deploy
```

```
NAME                              READY   STATUS    RESTARTS   AGE
pod/nginx-77b4fdf86c-bf5zs   1/1     Running   0          47s
pod/nginx-77b4fdf86c-cbvtp   1/1     Running   0          47s
```

```
pod/nginx-77b4fdf86c-k9bnm   1/1     Running   0          47s

NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx   3/3     3            3           47s
```

2. Convert Deployment to Rollout

Now we want to use the deployment definition to reference it in a new rollout:

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deployment
  workloadRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {duration: 10s}

EOF
```

Note the field **`workloadRef`**, which references the **`nginx-deployment`** resource.

As a result, we have 6 nginx instances running, 3 managed by our vanilla deployment, 3 by the newly created rollout:

```
kubectl get ro,deploy,po
```

```
NAME                                  DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
rollout.argoproj.io/nginx-rollout   3         3         3            3
101s

NAME                                READY   UP-TO-DATE   AVAILABLE
AGE
```

```
deployment.apps/nginx-deployment   3/3     3            3
2m3s
deployment.apps/spin               0/1     1            0
28h

NAME                                    READY   STATUS
RESTARTS         AGE
pod/nginx-deployment-66fb7f764c-8vhtz   1/1     Running             0
2m3s
pod/nginx-deployment-66fb7f764c-h5f2n   1/1     Running             0
2m3s
pod/nginx-deployment-66fb7f764c-sp4qr   1/1     Running             0
2m3s
pod/nginx-rollout-6d7df6cfcb-4kmmz      1/1     Running             0
101s
pod/nginx-rollout-6d7df6cfcb-cmtwq      1/1     Running             0
101s
pod/nginx-rollout-6d7df6cfcb-hjxxx      1/1     Running             0
101s
```

3. Scale Down Deployment

To finish the migration, we now need to manually scale down the deployment:

```
$ kubectl scale deployment/nginx-deployment --replicas=0
```

Congratulations, this leaves you with an up-and-running workload, managed by a rollout resource!

***NOTE**: In future versions of Argo Rollouts, the step of scaling down the deployment once referenced by the Rollout resource will be taken over by the Argo Rollout controller. A special **`scaleDown`** parameter will exist that enables administrators to specify how the deployment should be scaled down (**`never`**, **`onsuccess`**, **`progressively`**).*

This course is focusing on v1.6.4 of Argo Rollouts. We nevertheless consider it an important upcoming feature for migrating existing deployments and therefore want to mention it here.

4. Clean Up Resources

Make sure to leave the cluster nice and clean:

```
$ kubectl delete rollout nginx-rollout

rollout.argoproj.io "nginx-rollout" deleted

$ kubectl delete deployment nginx-deployment
```

```
deployment.apps "nginx-deployment" deleted
```

Lab 6.1 - Setting Up Event Triggers with Argo

In this chapter, we begin our hands-on journey with Argo Events, starting from setting up its basic components and triggering simple workflows with HTTP requests. After a theoretical overview, the first lab focuses on practical implementation, allowing users to familiarize themselves with Argo Events by using simple curl commands to initiate workflows. Building on this foundational knowledge, the chapter advances to a lab that integrates Argo Events with an external system, Apache Pulsar. This step-by-step exploration not only enhances understanding of Argo Events in real-world scenarios but also demonstrates its capability to seamlessly integrate with other technologies, showcasing its versatility in managing event-driven architectures in cloud-native environments.

Objective

- Set up a laboratory environment that demonstrates the integration and functionality of Argo Events and Argo Workflows.
- Install and configure the necessary components to trigger a workflow through an event.

Prerequisites

- A Kubernetes cluster is available.
- kubectl is installed and configured to communicate with your Kubernetes cluster.
- Internet access for downloading necessary files and packages.

Install and Use Argo Events

1. Installing Argo Workflows

Because we trigger a workflow we need to install Argo Workflows first. For that create the respective namespace “argo” and apply the installation YAML to the new namespace:

```
kubectl create namespace argo
```

```
kubectl apply -n argo -f
https://github.com/argoproj/argo-workflows/releases/download/v3.5.2/in
stall.yaml
```

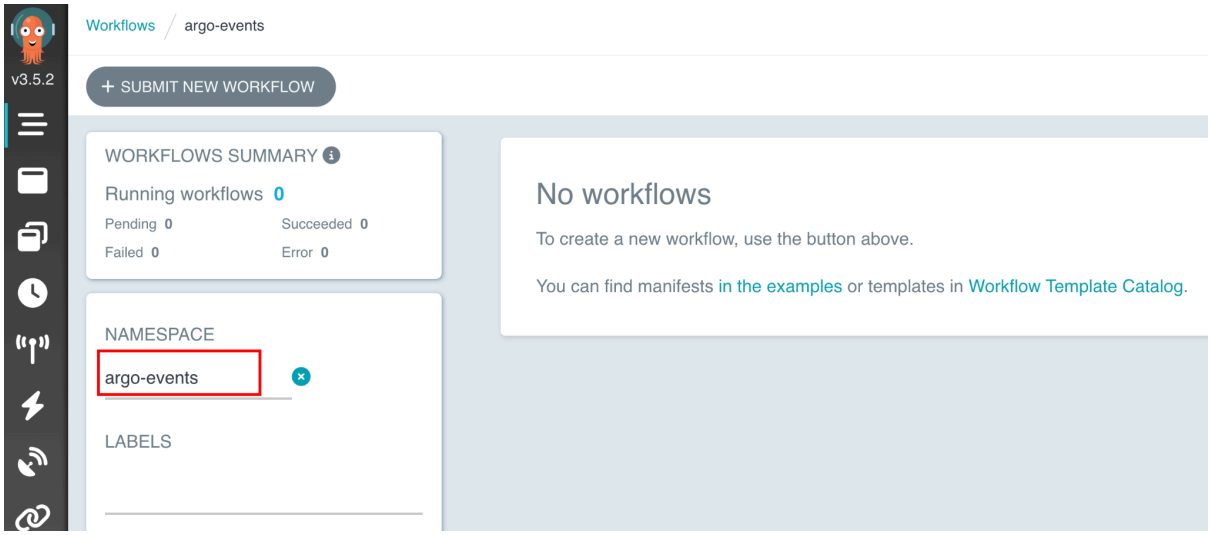
In the next step, we want to access Argo Workflows through the UI. For that we need to change the authentication mode of the Argo Server to “server”. It ensures that the server can properly authenticate requests:

```
kubectl patch deployment argo-server --namespace argo --type='json'
-p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/args", "value": ["server",
"--auth-mode=server"]}]'
```

Lastly port forward the Argo server deployment to access it on https://localhost:2746 locally:

```
kubectl -n argo port-forward deployment/argo-server 2746:2746
```

You should see the following screen:



Start screen of Argo Workflows

Make sure to select “argo-events” as the namespace.

2. Install Argo Events and the Needed Components

Creates a new namespace named 'argo-events', which is used to separate and manage the resources related to Argo Events. After that install Argo Events in your cluster by applying the installation manifest from the given URL:

```
kubectl create namespace argo-events
```

```
kubectl apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/manifest
s/install.yaml
```

The next command applies a validating webhook for Argo Events. Validating webhooks are used to ensure that incoming requests to the Kubernetes API server are valid:

```
kubectl apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/manifest
s/install-validating-webhook.yaml
```

For setting up a native EventBus in the 'argo-events' namespace, which handles event transportation in Argo Events, apply the configuration with this command:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/eventbus/native.yaml
```

Next we need to define an EventSource configuration that listens for webhook events in Argo Events, apply the following configuration using this command:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/event-sources/webhook.yaml
```

For the Sensor to properly interact with Kubernetes resources, apply the necessary RBAC policies:

```
kubectl apply -n argo-events -f
https://raw.githubusercontent.com/argoproj/argo-events/master/examples
/rbac/sensor-rbac.yaml
```

Similarly, apply RBAC policies for Workflows to ensure they have the necessary permissions in Kubernetes:

```
kubectl apply -n argo-events -f
https://raw.githubusercontent.com/argoproj/argo-events/master/examples
/rbac/workflow-rbac.yaml
```

Set up a Sensor to trigger workflows based on webhook events by applying this Sensor configuration:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/sensors/webhook.yaml
```

Expose the event-source pod via port forwarding to consume requests over HTTP:

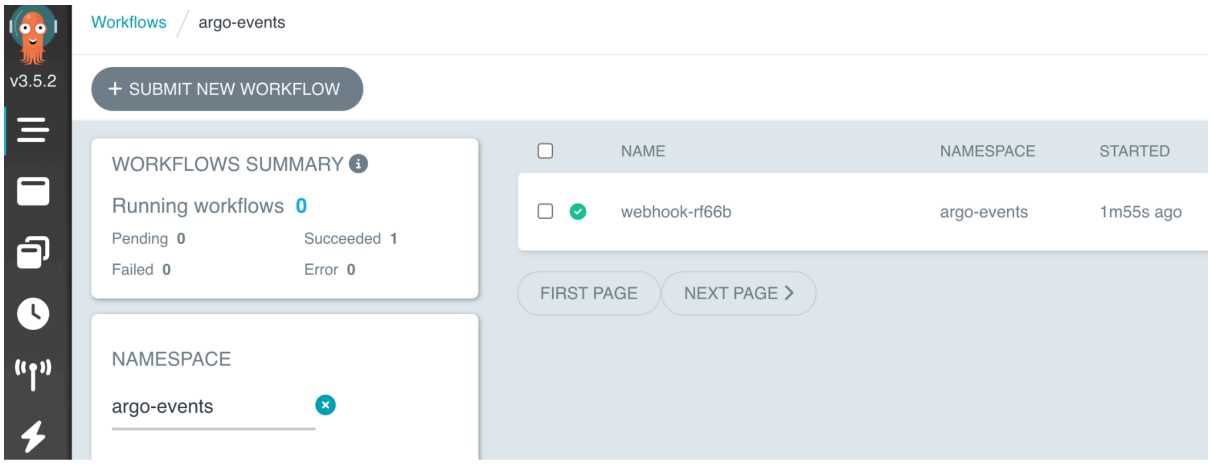
```
kubectl -n argo-events port-forward $(kubectl -n argo-events get pod
-l eventsource-name=webhook -o name) 12000:12000 &
```

Finally, simulate an external event that triggers the workflow. Send a test webhook event to the Event Source with this curl command:

```
curl -d '{"message":"this is my first webhook"}' -H "Content-Type:
application/json" -X POST http://localhost:12000/example
```

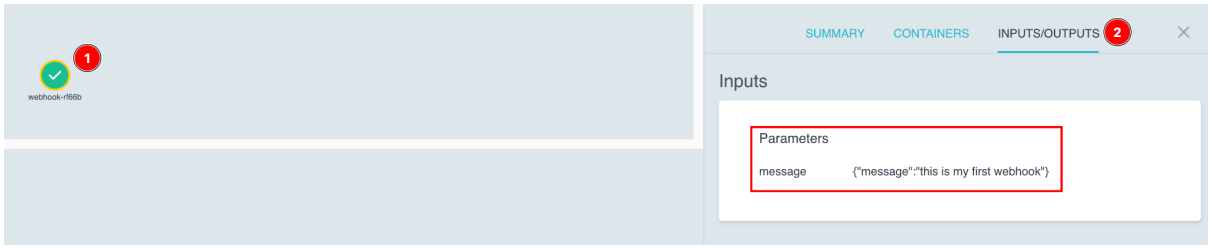
Refresh the UI of Argo Workflows. You should see a new workflow that is processing.

Refresh again after a few minutes and you should see the following screen with a completed workflow:



Workflow is shown in Argo Workflows UI

Click on the name of the workflow (e.g., webhook-rf66b). In the new window select the webhook (1), select Inputs/Outputs (2) and under parameters you see the message of the curl we have sent before.



Message of curl shown in the workflow

We successfully implemented a lab setup integrating Argo Workflows and Argo Events within a Kubernetes environment, demonstrating the efficiency of event-driven automation combined with workflow management.

The key achievement was using a webhook event to initiate an Argo Workflow, showcasing the system's capability to respond to events dynamically.

In our next lab, we plan to extend this setup to connect with an external system.

Lab 6.2 - Integrating Argo Events with External Systems

This lab is designed to expand your experience with Argo Events by integrating it with Apache Pulsar. After having previously triggered Argo Events workflows using a simple HTTP request, you will now use Pulsar, a distributed messaging system, as a different event source.

Objectives

- Gain practical experience in setting up and triggering workflows in Argo Events using messages from external systems.

Prerequisites

- A Kubernetes cluster is available with the components installed from Lab 6.1.
- kubectl is installed and configured to communicate with your Kubernetes cluster.
- Internet access for downloading necessary files and packages.

Use Apache Pulsar with Argo Events

1. Triggering a Workflow with Pulsar

Deploy Apache Pulsar in your cluster with:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/lftraining/LFS256-code/main/argoevents/pulsar.yaml
```

Check the status of the Pulsar pod and note the name:

```
kubectl get pods -n argo-events
```

Next, we need to port forward the Pulsar pod to enable direct communication between your local machine and the Pulsar service running in the Kubernetes cluster. This step is crucial for Argo Events to interact with Pulsar for triggering workflows. We do that with the following command:

```
kubectl -n argo-events port-forward POD NAME OF PULSAR 6650:6650
```

Replace **POD NAME OF PULSAR** with the actual name of the Pulsar pod obtained from the **kubectl get pods** command.

Set up the event source for Argo Events to listen to Pulsar messages. This configures Argo Events to connect and listen to Pulsar:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/event-sources/pulsar.yaml
```

Deploy the sensor for reacting to Pulsar events. This sets up the actions to be taken in response to events detected by Argo Events:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/sensors/pulsar.yaml
```

Now, everything is set up to trigger the event. To interact with the Pulsar pod, use:

```
kubectl -n argo-events exec -it NAME OF PULSAR POD -- /bin/bash
```

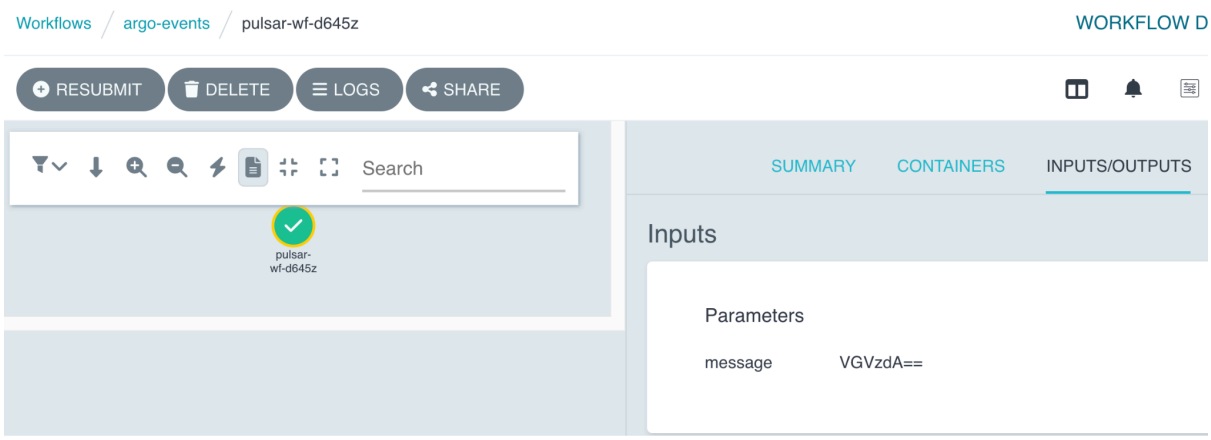
Inside the Pulsar pod, navigate to the bin directory and send a test message:

```
cd bin
```

```
./pulsar-client produce test --messages "Test"
```

2. Inspecting the Triggered Workflow

After sending the "Test" message via Pulsar, it triggers an Argo workflow. In the Argo UI, you can see the message in the workflow the same as in the first lab.



Message of Pulsar shown in the workflow

However, this message is encoded in Base64, a common practice in Apache Pulsar for efficient and reliable data serialization and transmission. To read the message correctly, you'll need to decode it from Base64.

In this lab, we focused on the integration of Argo Events with Apache Pulsar, demonstrating how Argo Events can be configured to interact with external messaging systems. This exercise highlights Argo Events' versatility in integrating with different external systems, enhancing its capability to manage complex, event-driven architectures in cloud-native environments.